WEb-Attacks Writeup | Insecure Direct Object References (IDOR)

HackTheBox

Written by: Alexander Sapo

Mass IDOR Enumeration

Question

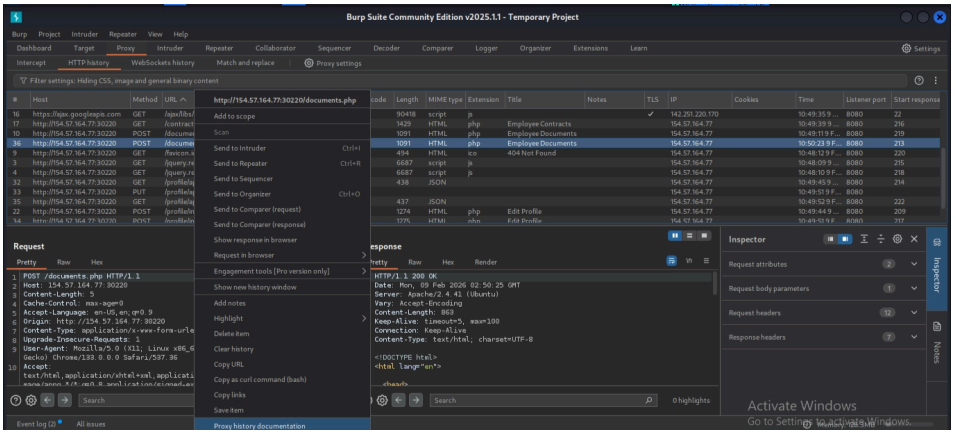
Repeat what you learned in this section to get a list of documents of the first 20 user uid's in `/documents.php`, one of which should have a `.txt` file with the flag.

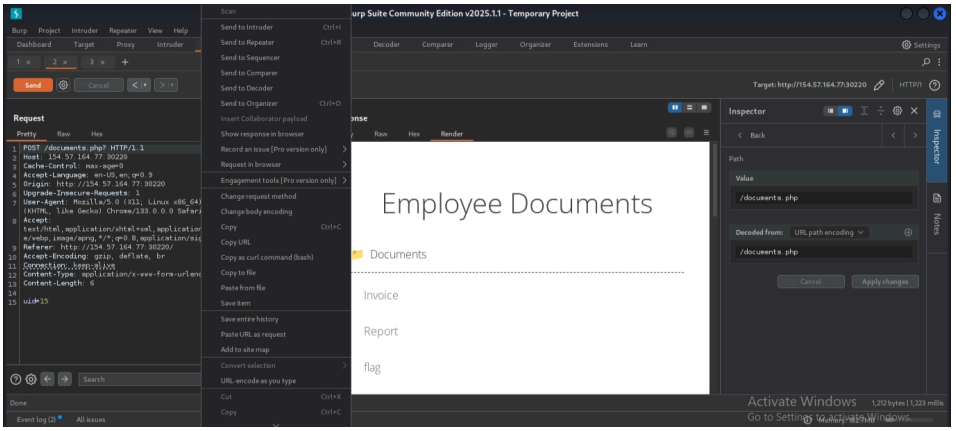
Flag: `HTB{4ll_f1l35_4r3_m1n3}`

Process

Fire up Burp Suite.

Right-click and send `POST /documents.php` to Intruder.





Below, you will see a parameter:

`uid`

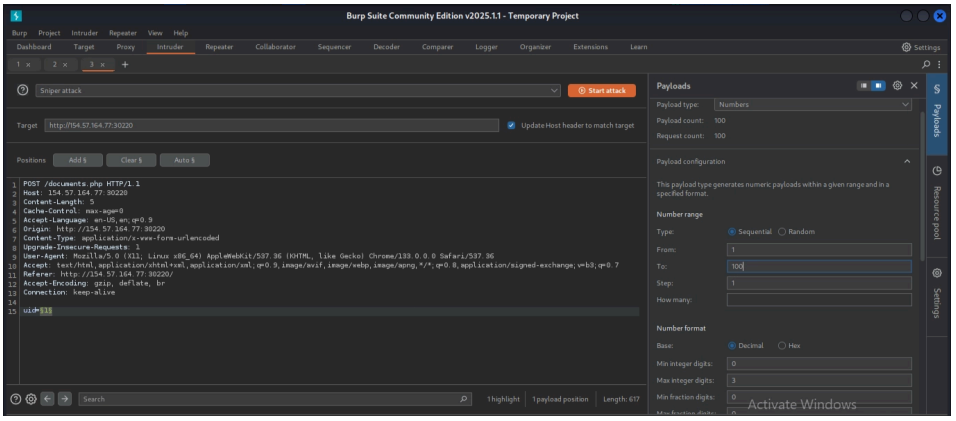
Automate using Intruder by iterating through users.

Select the value of `uid`, then right-click and add Position.

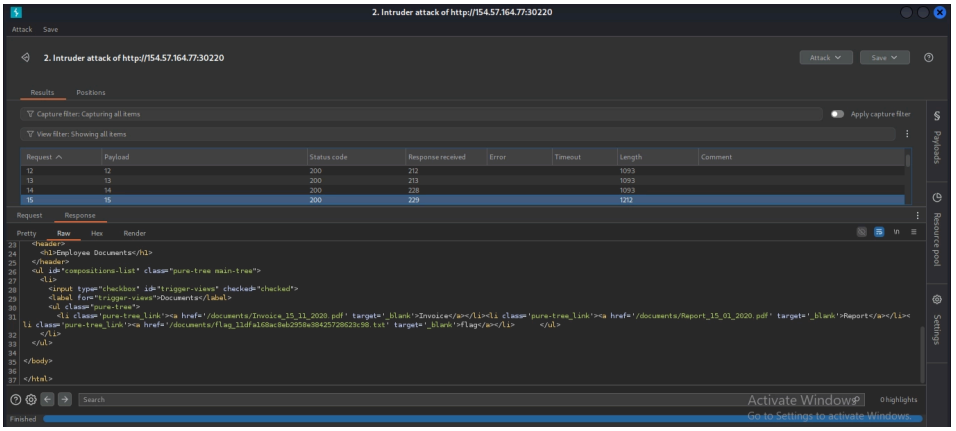
In the Payloads section, select payload type **Numbers**.

Set from **1 to 20** as mentioned in the challenge.

Start attack.



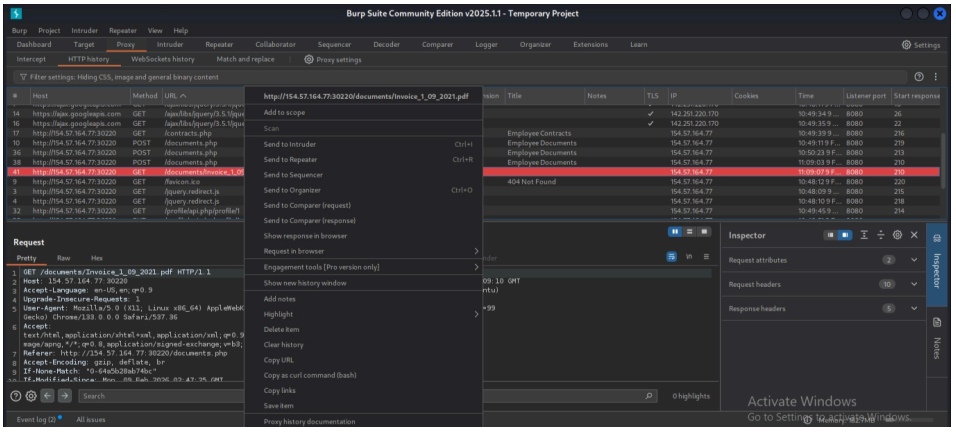
Check the responses from our requests.



As we check each user, we finally find a file name containing the flag in **user 15 Documents**.

The backend never validated who is accessing and also trusted the parameter from the client side without validation.

Now let's proceed to HTTP history and look for:



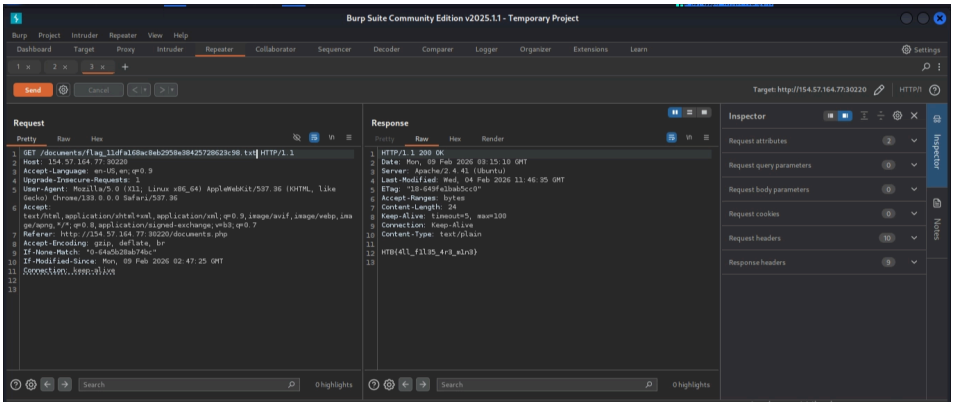
`/document/invoice_1_09_2021.pdf`

(GET HTTP method)

This is when we try to access our files from our documents.

Earlier, we saw the files of user 15 and the flag name.

Now with the backend having no validations, this allows us to access user 15 files in:
`/document/<flag file name>`



Bypassing Encoded References

Question

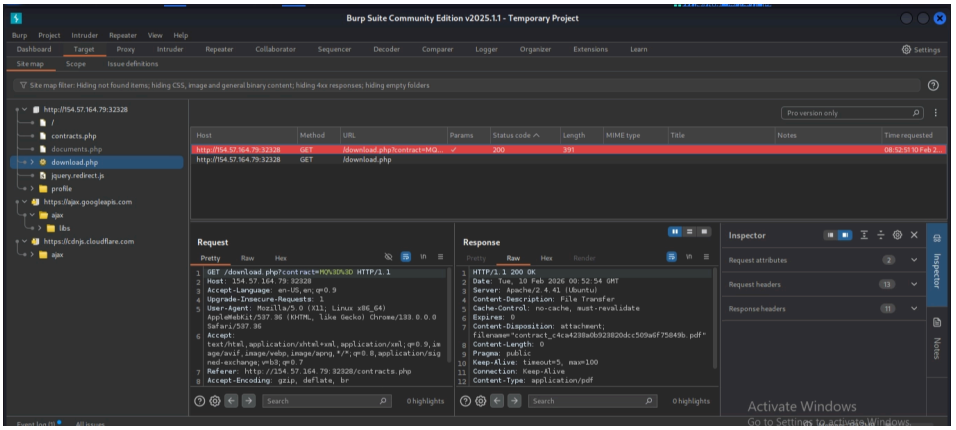
Try to download the contracts of the first 20 employee, one of which should contain the flag, which you can read with `cat`. You can either calculate the `contract` parameter value, or calculate the `.pdf` file name directly.

Flag: `HTB{h45h1n6_1d5_w0n7_570p_m3}`

Process

Fire up Burp Suite.

Go to Sitemap in Target.

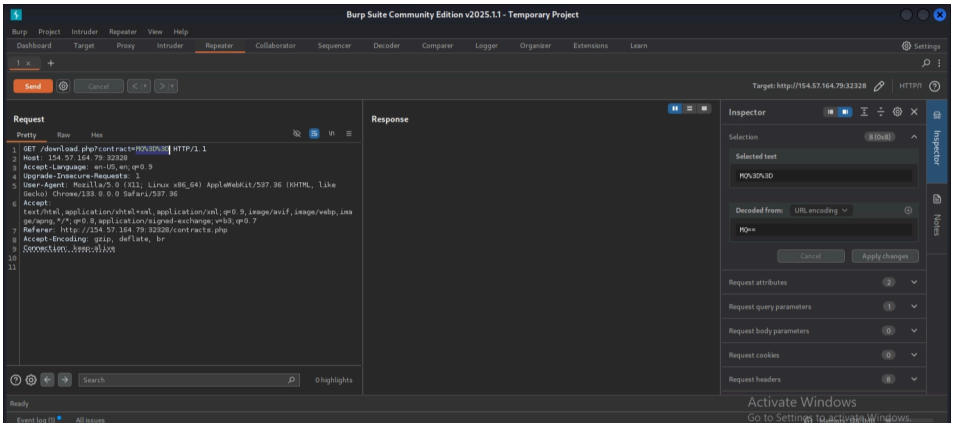


As we have intercepted the web, we can see two URLs:

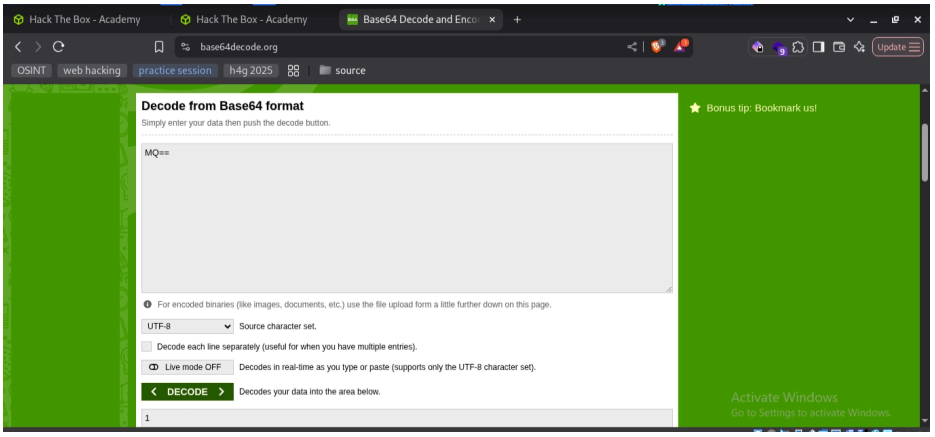
```
/download.php?contract=<urlencoded value>
/download.php
```

Right-click → Send to Repeater.

URL-decode and we can see the value.



`MQ=` when decoded is `1`, which is Base64.

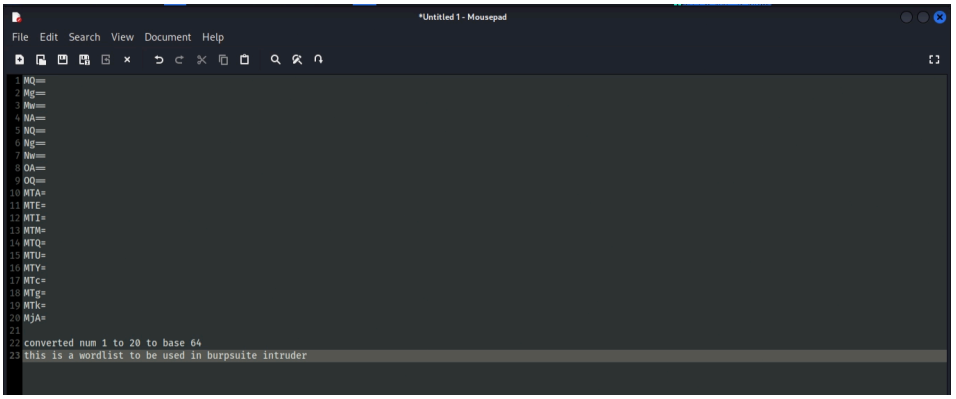


This is a way to obfuscate the value, yet the issue is it can easily be decoded.

It is also vulnerable to IDOR since the backend has no access control in place.

Let's create a txt list of encoded numbers 1 to 100 to be sure.

Base64-encode numbers and save as txt.



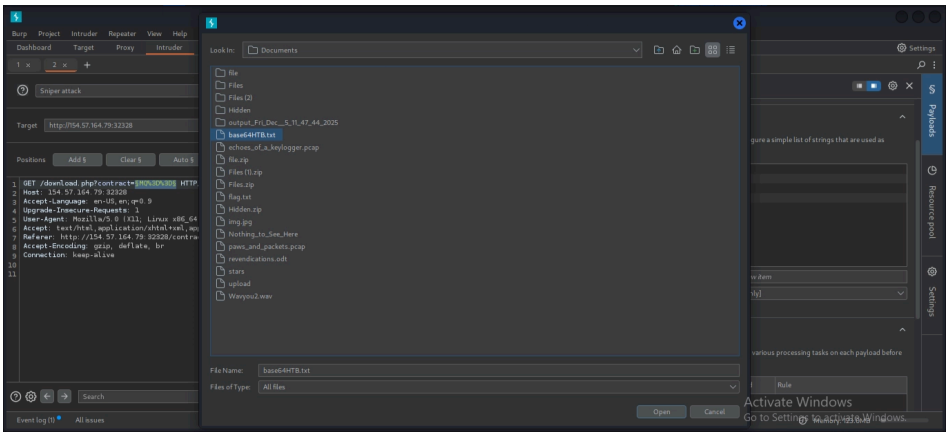
Then proceed to sending the request we got to Intruder:

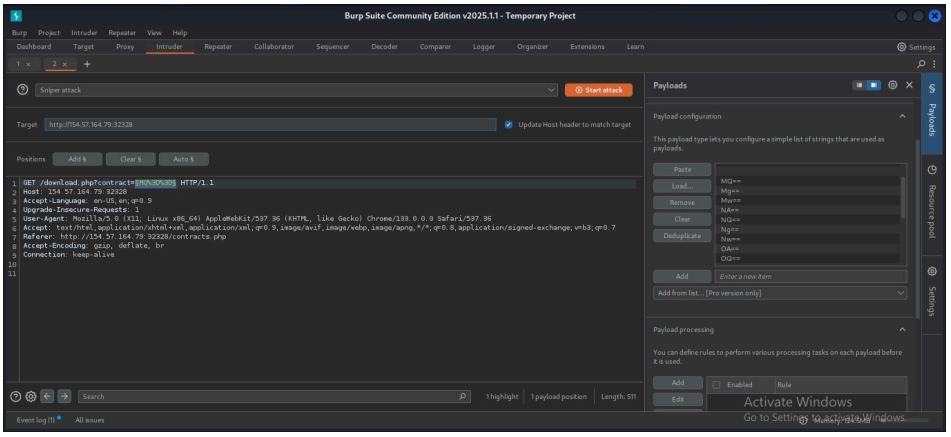
```
GET /download.php?=<selected value>
```

Select its value → right-click → Add Position.

Now proceed to Payloads section.

Payload configuration → add our Base64 encoded wordlist.



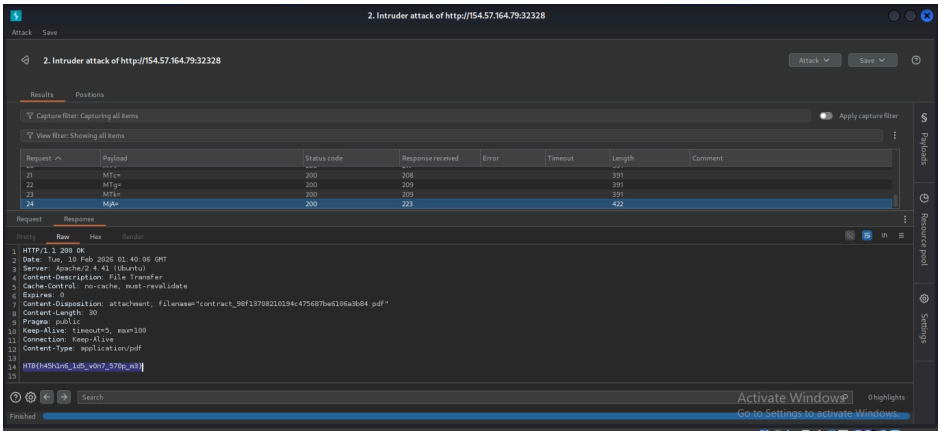


This automates checking each user.

Click Start Attack.

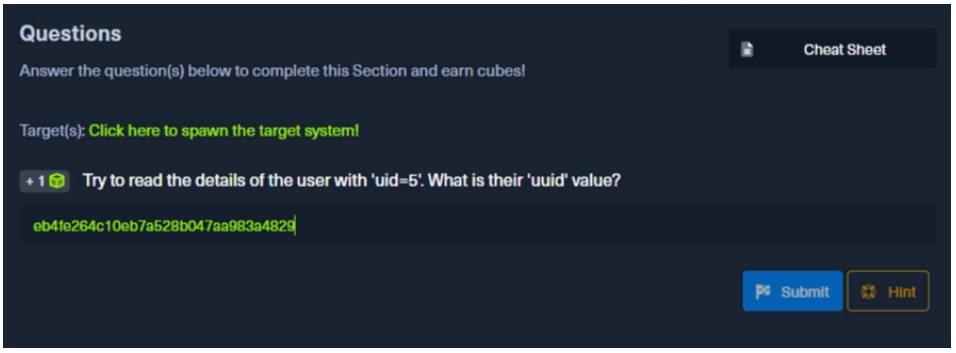
We then proceed to the responses of each request.

As we check, we find `MjA=` which is `24`, and here we see the flag.



IDOR in Insecure APIs

Question



Try to read the details of the user with `uid=5`. What is their `uuid` value?

Flag: `eb4fe264c10eb7a528b047aa983a4829`

Process

HTTP verb tampering plays a role in IDOR insecure API.

There is existing validation in the backend and proper configurations,

but the issue occurs when the HTTP method is changed.

In specific methods, there was no validation in the backend

and also no API configuration.

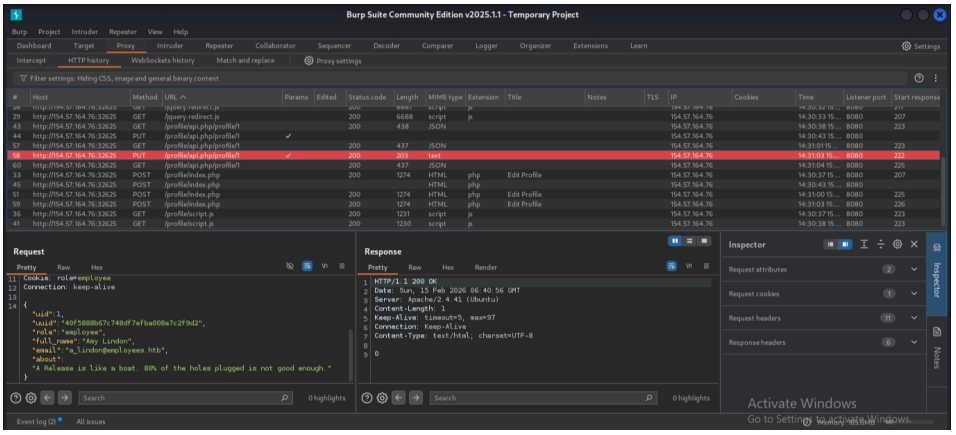
Fire up Burp Suite.

Open browser → click Edit Profile → then Update to capture it.

Paste target in browser → proceed to Proxy → HTTP history.

We find:

`/profile/api.php/profile1`



Method: PUT

As mentioned, to view data we use the GET method.

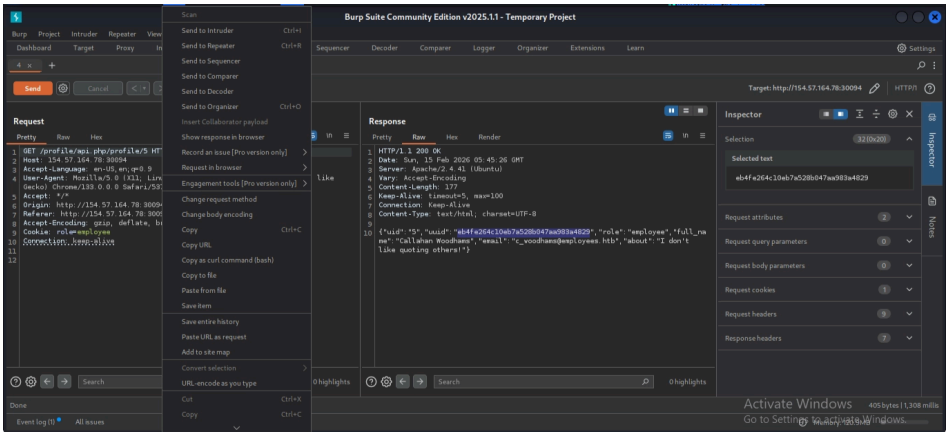
Right-click → Send to Repeater.

Change profile from `1` to `5` as in the challenge.

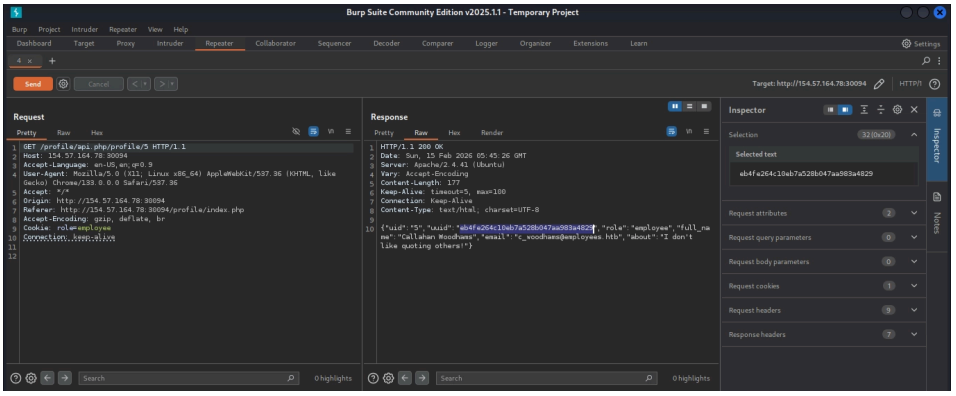
Also change `uuid` parameter in JSON,

which is client-side and modifiable.

Right-click → Change request method → GET → Send.



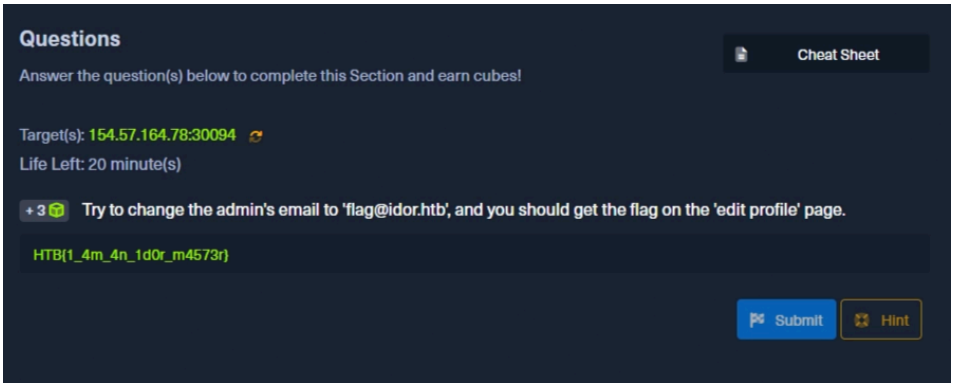
And there we go:




```
{"uid":"5","uuid":"eb4fe264c10eb7a528b047aa983a4829","role":"employee","full_name":"Callah
an Woodhams","email":"c_woodhams@employees.htb","about":"I don't like quoting others!"}
```

Chaining IDOR Vulnerabilities

Question



Try to change the admin's email to `flag@idor.htb`, and you should get the flag on the edit profile page.

Flag: `HTB{1_4m_4n_1d0r_m4573r}`

Process

The first challenge or attack allowed us

to view other users' information.

Now we proceed to attempt modifying the admin's

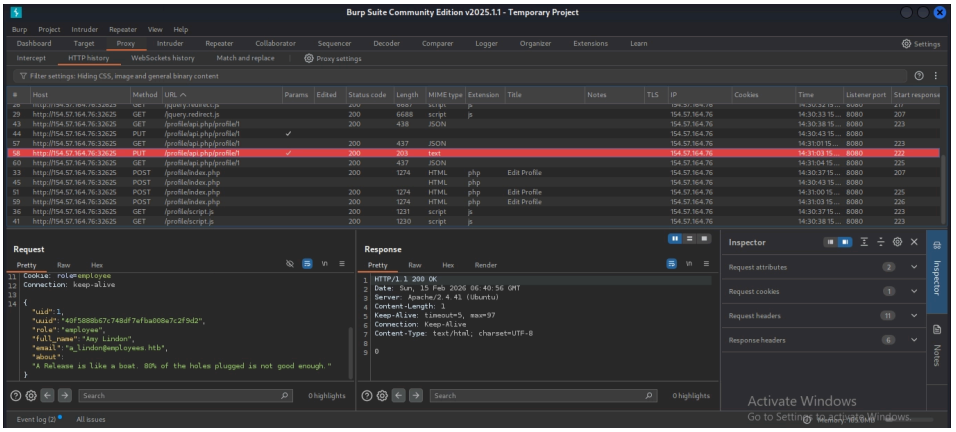
email address as requested by the challenge.

Fire up Burp Suite.

Send to Repeater the same request we got earlier:

```
PUT /profile/api.php/profile/1
```

Right-click → Change HTTP method → GET



to view user info.

Then right-click → Send to Intruder

to automate iterating through users to find admin.

```
GET /profile/api.php/profile/1
```

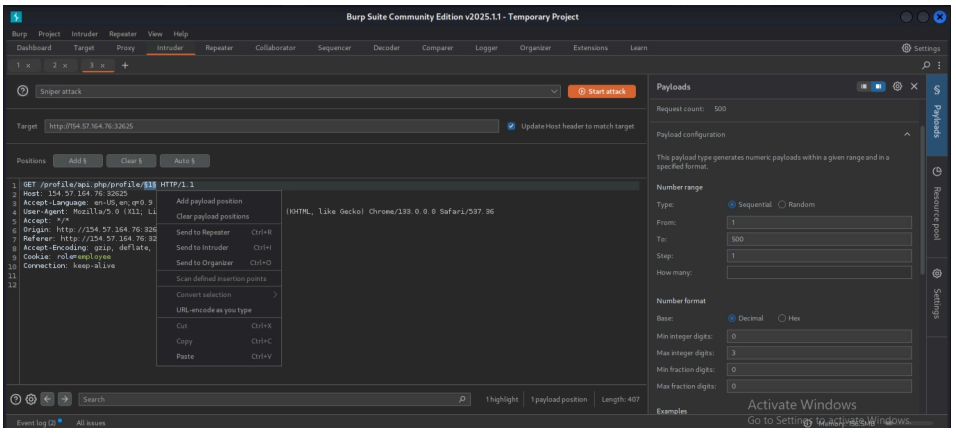
Select `1` → right-click → Add Position.

Proceed to Payloads section:

Payload type: Numbers

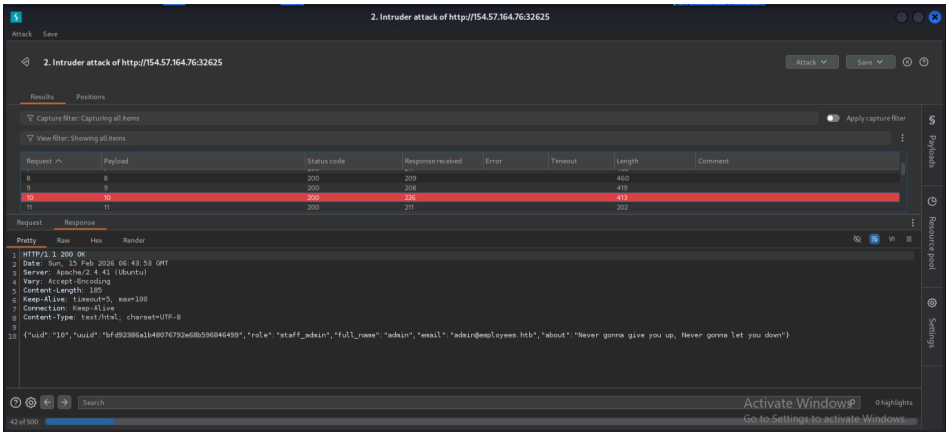
Range: 1–50

Click Start Attack.



A pop-up window opens.

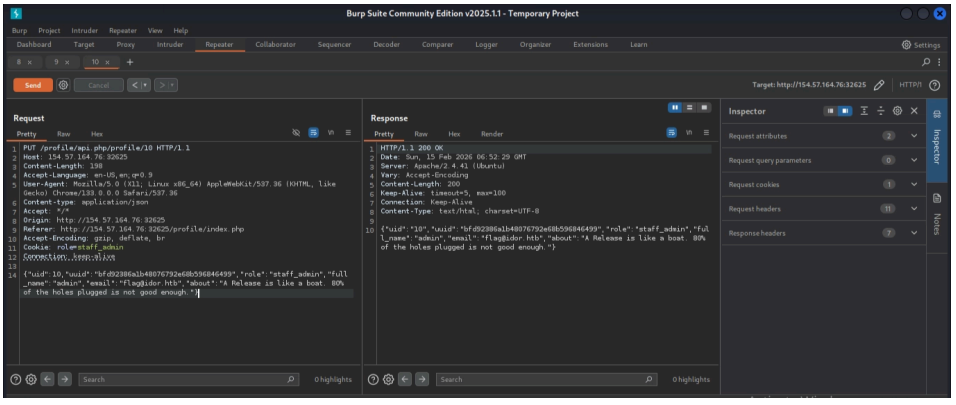
Check each response.



We find **user 10** is admin.

Copy UUID.

Go back to Repeater and use:




```
PUT /profile/api.php/profile/10
```

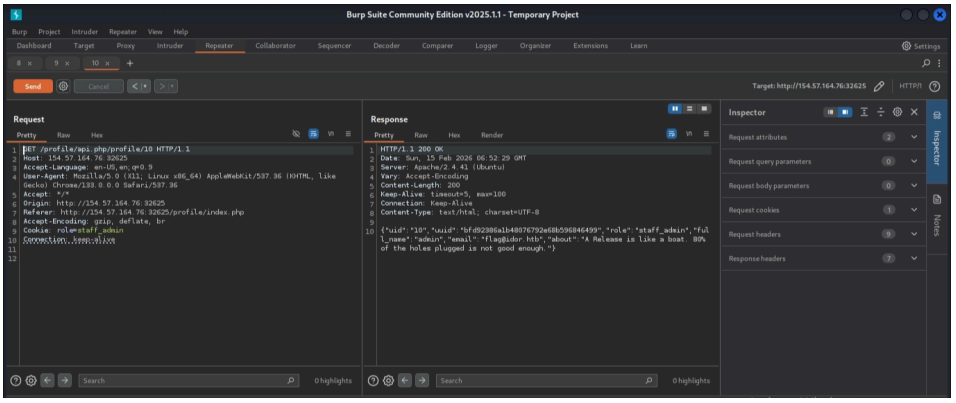
Replace uid=10 UUID=bfd92386a1b48076792e68b596846499 with the admin UUID we found.

> {"uid":10,"uuid":"bfd92386a1b48076792e68b596846499","role":"staff_admin","full_name":"admin","email":"flag@ido
> Release is like a boat. 80% of the holes plugged is not good enough."}As mentioned in the challenge,

we change email to `flag@idor.htb`.

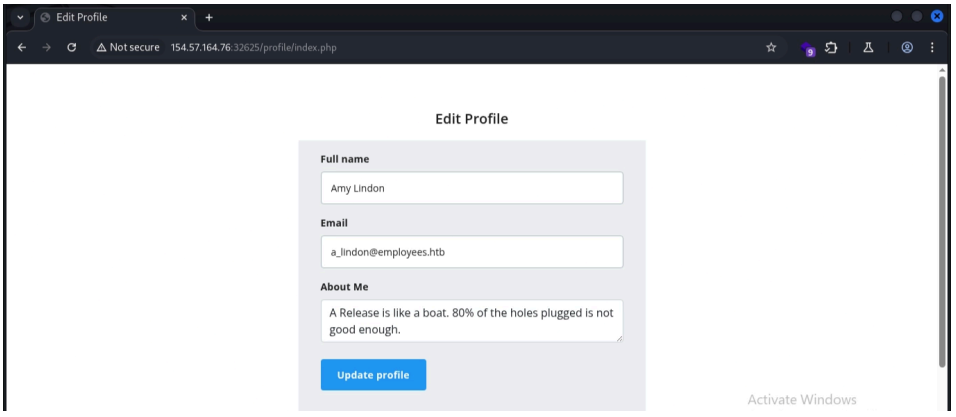
Send request.

Then change HTTP method to GET



to verify the change occurred.

Go to browser → Edit Profile page.



Flag visible at:

```
/profile/index.php
```